

```

Speech2Speech agent
# You may need to add your working directory to the Python path. To do so,
uncomment the following lines of code
# import sys
# sys.path.append("/Path/to/directory/agent-framework") # Replace with
your directory path

# Besser Agentic Framework Multilingual speech-to-speech example agent

# imports
import logging
import base64

from baf.core.agent import Agent
from baf.core.session import Session
from baf.exceptions.logger import logger

from baf.nlp.llm.llm_openai_api import LLMOpenAI
from baf.nlp.speech2text.openai_speech2text import OpenAISpeech2Text

from baf.nlp.speech2text.luxasr_speech2text import LuxASRSpeech2Text
from baf.nlp.text2speech.openai_text2speech import OpenAIText2Speech
from baf.nlp.text2speech.piper_text2speech import PiperText2Speech

from baf.core.file import File
from baf.library.transition.events.base_events import ReceiveFileEvent,
ReceiveMessageEvent
from baf.library.transition.events.base_events import ReceiveJSONEvent

from baf.core.processors.audio_language_detection_processor import
AudioLanguageDetectionProcessor

# Configure the logging module (optional)
logger.setLevel(logging.INFO)

# Create the agent
agent = Agent('Multilingual Speech-to-Speech Agent')

# Load agent properties stored in a dedicated file
agent.load_properties('config.yaml')

# Define the platform your agent will use
websocket_platform = agent.use_websocket_platform(use_ui=True)

# Define STT and TTS Models
stt = OpenAISpeech2Text(agent=agent, model_name="whisper-1", language=
"en")
stt2 = OpenAISpeech2Text(agent=agent, model_name="gpt-4o-mini-transcribe")
tts = OpenAIText2Speech(agent=agent, model_name="gpt-4o-mini-tts",
language="en", voice="coral")
tts2 = OpenAIText2Speech(agent=agent, model_name="gpt-4o-mini-tts",
language="fr", voice="ash")
stt_lux = LuxASRSpeech2Text(agent=agent, language="lb")

```

```

piper = PiperText2Speech(agent, language="lb")

# Create the LLM
gpt = LLMOpenAI(
    agent=agent,
    name='gpt-4.1',
    parameters={},
    num_previous_messages=100,
    global_context='You are a helpful assistant. Always match and answer in
the language the user is speaking to you. '
    'Keep your answers concise and to the point. Do not use
any formatting or bullet points.',
)

# Define processor (for spoken language recognition)
process = AudioLanguageDetectionProcessor(agent=agent,
transcription_model=stt2, llm_name='gpt-4.1')

# States
initial_state = agent.new_state('initial_state', initial=True)
awaiting_state = agent.new_state('awaiting_state') # for awaiting user
input
sts_state = agent.new_state('sts_message_state') # for messages and speech
sts_file_state = agent.new_state('sts_file_state') # for audio files
uploaded through the UI

# STATES BODIES' DEFINITION + TRANSITIONS

def initial_body(session: Session):
    session.set("user_language", "en") # Set default user language to
English
    answer = gpt.predict(
        f"You are a helpful assistant. Start the conversation with a short
(2-15 words) greetings message. Make it original.")
    session.reply(answer)

initial_state.set_body(initial_body)
initial_state.go_to(awaiting_state)

def awaiting_body(session: Session):
    pass

awaiting_state.set_body(awaiting_body)
awaiting_state.when_file_received(allowed_types=("audio/wav", "audio/mpeg",
"audio/mp4", "text/plain")).go_to(
    sts_file_state) # Only Allow Wav, MP3, MP4 files
awaiting_state.when_event(ReceiveJSONEvent()).go_to(sts_state) # when
Audio is received through the UI
awaiting_state.when_no_intent_matched().go_to(sts_state)

def stt_message_body(session: Session):
    # only transcribe message if the user spoke

```

```

        if isinstance(session.event, ReceiveJSEvent) or
isinstance(session.event, ReceiveMessageEvent):
            session.reply("User: " + session.event.message)
            answer = gpt.chat(session)
            websocket_platform.reply_speech(session, answer)
            session.reply(answer)

sts_state.set_body(stt_message_body)
sts_state.go_to(awaiting_state)

# Execute when a file is received
def stt_file_body(session: Session):
    # get user language
    lang = session.get("user_language", "en")
    # access STT system based on language mapping
    s2t = session._agent._nlp_engine._language_to_speech2text_module[lang]
    event: ReceiveFileEvent = session.event
    file: File = event.file

    # Determine MIME type
    ext = file.name.lower()
    # do only for text files
    if ext.endswith(".txt"):
        mime_type = "text/plain"
    elif ext.endswith(".wav"):
        mime_type = "audio/wav"
    elif ext.endswith(".mp3"):
        mime_type = "audio/mpeg"
    elif ext.endswith(".m4a"):
        mime_type = "audio/mp4"
    else:
        mime_type = "application/octet-stream"

    # only when audio files are uploaded
    if not mime_type == "text/plain":

        # convert file to byte representation
        base64_content = file._base64
        # Decode the base64 string into bytes
        file_bytes = base64.b64decode(base64_content)
        # add to logger
        logger.info(f"Successfully decoded {len(file_bytes)} bytes.")
        # for the LuxASR model, we need to indicate the MIME Type
        if lang == "lb":
            # call LuxASR Speech2Text and get transcription
            text = s2t.speech2text(file_bytes, mime_type)
        else:
            text = s2t.speech2text(file_bytes)
        session.reply("User: " + text)
        answer = gpt.predict(text)
        #session.reply(answer)

```

```
        file_text = answer
    else:
        # convert file to byte representation
        base64_content = file._base64
        # Decode the base64 string into text
        file_text = base64.b64decode(base64_content).decode('utf-8')

    # call Speech2Text and get transcription
    session.reply(file_text)
    websocket_platform.reply_speech(session, file_text)

sts_file_state.set_body(stt_file_body)
sts_file_state.go_to(awaiting_state)

if __name__ == '__main__':
    agent.run()
```